Client Server Architectures (CSA) Course Work

Module: 5COSC022W Client-Server Architectures
Assignment Type: Individual
Submission Date: 2026.04.24

Student ID: 20231264 / w2120198
Student Name: Ranuga Disansa Belpa Gamage
Student Email: ranuga.20231264@iit.ac.lk

Table of Contents

Question 1 ..... ***3***

Question 2 ..... ***4***

Question 3 ..... ***6***

Question 4 ..... ***7***

Question 5 ..... ***8***

Question 6 ..... ***9***

Question 7 ..... ***10***

Question 8 ..... ***11***

Question 9 ..... ***12***

Table of Figures

Figure 1 3
Figure 2 4
Figure 3 5
Figure 4 6
Figure 5 6
Figure 6 7
Figure 7 8
Figure 8 9
Figure 9 10
Figure 10 11
Figure 11 12

Question 1

Q1. In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

JAX-RS resource classes default to a per request lifecycle in Jersey: a fresh instance is constructed for every incoming request unless the class carries the Singleton scope annotation, and no resource in this project does. Instance fields on the resources are therefore thread confined, and the genuinely shared state lives in the DAO classes.

Each DAO holds a static final collection and guards every read and write with a synchronized block on that collection, so iteration cannot race with a concurrent write. The Room and Sensor DAOs wrap synchronized lists. The Sensor Reading DAO uses a plain hash map keyed by sensor id under the same locking idiom.

The per DAO locks are enough when a request touches one store. Room delete and sensor create each touch two stores in the same request, and the locks on separate DAOs do not talk to each other. Two threads running at the same time can therefore leave the system in a state where a sensor still points at a room that has just been deleted.

The rule we want to keep is simple: a room's sensor id list should be empty only when no sensor references that room. Two orderings break this rule. A concurrent delete can see the list as empty and finish between a concurrent creates check and it append. The reverse can also happen, with a create slipping between a delete's empty check and its actual removal.

Both services therefore acquire a dedicated shared monitor declared on the room DAO at the top of their critical section and hold it through the final write. The critical section inside the delete path is shown below.

```java
public boolean delete(String id) throws RoomNotEmptyException {
    synchronized (RoomDAO.LINK_LOCK) {
        Room existing = roomDAO.getById(id);
        if (existing == null) {
            return false;
        }
        if (existing.getSensorIds() != null && !existing.getSensorIds().isEmpty()) {
            throw new RoomNotEmptyException(
                    "Room '" + id + "' still has " + existing.getSensorIds().size()
                    + " active sensor(s) assigned and cannot be deleted.");
        }
        roomDAO.delete(id);
        return true;
    }
}
```

Figure 1

Question 2

Q2. Why is the provision of "Hypermedia" (links and navigation within responses) considering a core feature of advanced REST design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

HATEOAS (Hypermedia As The Engine Of Application State) is the practice of including navigational links inside API responses so that a client can discover the available state transitions dynamically rather than hard coding them from documentation. The specification describes it as a core feature of advanced REST design because it completes the separation between the client's model of the service and the server's URI scheme. When a response carries the links a client needs for its next request, the server is free to move endpoints, rename path segments, or introduce a new version so long as the hypermedia in the responses stays internally consistent.

The benefit over static documentation is practical rather than theoretical. A PDF reference or a README sample URL is an external copy of the server's URI scheme that drifts the moment the code changes, whereas a live discovery response is produced by the same code that handles the requests and therefore cannot be out of sync with the code that produced it.

Hypermedia also lets the server express conditional availability through the presence or absence of links. Consider a hypothetical HATEOAS extension to the room response. Immediately after creation, room *LIB-301* has no sensors assigned, so the response could include a *delete* link alongside *self* and *sensors*:

```
{
  "id": "LIB-301",
  "name": "Library Reading Room",
  "capacity": 40,
  "sensorIds": [],
  "_links": {
    "self":    { "href": "/api/v1/rooms/LIB-301" },
    "sensors": { "href": "/api/v1/sensors?roomId=LIB-301" },
    "delete":  { "href": "/api/v1/rooms/LIB-301", "method": "DELETE" }
  }
}
```

Figure 2

Once a CO2 sensor is registered to that room, the server would drop the *delete* link from the response, reflecting the business rule that blocks deletion of a room with active sensors. A client rendering a management dashboard could hide the delete button based purely on the presence of that link, without needing to know the 409 rule exists. The same logic applies to sensors: an *ACTIVE* sensor carries a *post-reading* link, while the same sensor in *MAINTENANCE* omits it, so the client knows not to offer that action without a separate status check.

The Smart Campus API's discovery endpoint is the practical entry point for this pattern. *GET /api/v1* returns a JSON object with an *administrator* block, a *version* field, and a *resources* map that lists every top-level collection together with its absolute URI. A representative response looks like this:

```
{
  "apiName": "Smart Campus Sensor & Room Management API",
  "version": "1.0.0",
  "administrator": {
    "name": "Ranuga Disansa Gamage",
    "email": "ranuga.20231264@iit.ac.lk"
  },
  "resources": {
    "rooms": "http://localhost:8080/smart-campus-sensor-and-room-management-api/api/v1/rooms",
    "sensors": "http://localhost:8080/smart-campus-sensor-and-room-management-api/api/v1/sensors"
  }
}
```

Figure 3

The URIs are built at request time from *UriInfo.getBaseUriBuilder()* so the response reflects the runtime deployment without any hard coded values. A client bookmarks only the discovery URL and follows the *resources* map from there.

Question 3

Q3. When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client-side processing.

A list endpoint has two basic options: return just the identifiers and let the client fetch each record on demand or return the full object for every entry in one response.

Option 1: ID only list

The response is a flat array of strings:

```
["LIB-301", "LEC-101", "LAB-201"]
```

Figure 4

The payload is small, but a client that needs each room's name or capacity must then issue one further request per identifier. For a collection of 40 rooms that is 41 HTTP round trips in total. Each extra request carries its own dispatch and serialisation cost, so the bandwidth saving from the smaller initial response can quickly be outweighed by the aggregate round trip overhead.

Option 2: Full object list

Each entry carries all fields:

```
[
  {
    "id": "LIB-301",
    "name": "Library Quiet Study",
    "capacity": 40,
    "sensorIds": ["TEMP-001"]
  },
  {
    "id": "LEC-101",
    "name": "Lecture Hall A",
    "capacity": 120,
    "sensorIds": []
  }
]
```

Figure 5

A single request is enough to populate a full table view. The costs are a larger payload for clients that only need a subset of fields, and a change to any one room invalidates the entire list response in a cache.

The Smart Campus API's choice is Option 2 for rooms, but with a deliberate limit: the embedded *sensorIds* field is an ID only list rather than a list of full *Sensor* objects. A room carries the identifiers of the sensors it hosts, but the full sensor data lives at the */sensors* endpoint. That avoids duplicating sensor fields inside every room response and keeps each resource independently cacheable. The present API does not implement pagination or field selection because the specification does not require them.

Question 4

Q4. Is the *DELETE* operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same *DELETE* request for a room multiple times.

The *DELETE* operation is idempotent in this implementation. Idempotency means that applying an operation once has the exact same effect on server state as applying it multiple times. The distinction matters: it is a property of what the server state looks like after the request, not of which status code the client receives.

Tracing the three cases against *SensorRoom.deleteRoom* in *resource/SensorRoom.java*, which delegates to *service/RoomService.delete*:

1. **First call on a room with no sensors.** *roomService.delete(roomId)* looks up the entity, finds the *sensorIds* collection empty, removes the room from the in-memory list, and returns *true*. The resource converts that into *204 No Content*. The server state after this call contains no such room.
2. **Second identical call.** *roomService.delete(roomId)* fails its lookup because *roomDAO.getById(roomId)* returns *null*, and the service returns *false*. The resource maps the *false* return value to an inline *404 Not Found* response with an *ErrorMessage* body. The server state does not change: the room is still absent.
3. **Third, fourth, and further calls.** These behave exactly as call 2. The 404 body is deterministic and the store is untouched.

The observable sequence of status codes for the same request sent three times against a room that starts with no sensors is shown below.

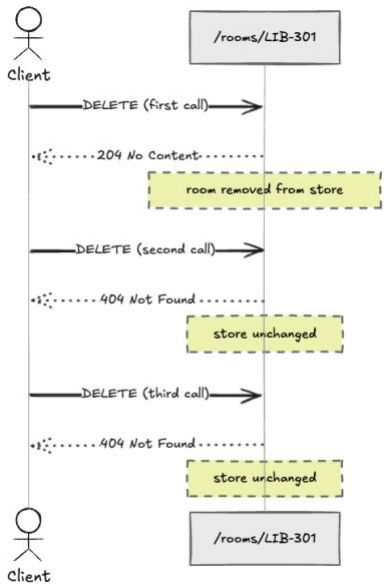

Figure 6

The status codes differ between call 1 and calls 2+, but the property required by idempotency is satisfied: after any number of identical *DELETE /rooms/{id}* requests, the room is absent. A client that retries a failed request converges on the same server state as one that succeeded on the first attempt.

The 409 Conflict case ("room still has sensors") does not threaten this guarantee. A 409 response leaves the room exactly where it was, so the server state is unchanged on every repetition. The response is also deterministic until some other request removes the dependent sensors, at which point the next *DELETE* succeeds and subsequent ones revert to the 404 pattern above. The operation therefore remains idempotent in the presence of the business rule.

Question 5

Q5. We explicitly use the *@Consumes(MediaType.APPLICATION_JSON)* annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as *text/plain* or *application/xml*. How does JAX-RS handle this mismatch?

@Consumes(MediaType.APPLICATION_JSON) declares which representations a resource method is willing to accept. The declaration is consumed by Jersey's dispatch stage, not by the method body, the runtime inspects the request's *Content-Type* header and matches it against every candidate method on the resource before invoking any of them.

If no method matches (the client sent *text/plain* or *application/xml* against a JSON only endpoint), the dispatch stage throws *javax.ws.rs.NotSupportedException*, a subtype of *WebApplicationException*. The HTTP response is **415 Unsupported Media Type** with the method body never executed, so any downstream validation or persistence logic is irrelevant, the client has been rejected at the contract boundary.

In this project the 415 is produced by Jersey and routed through *GenericExceptionMapper*, which preserves the *WebApplicationException*'s intended status and wraps the error in the standard JSON envelope:

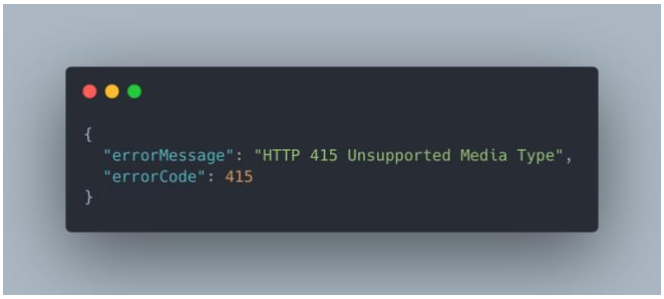

Figure 7

@Produces is the mirror of *@Consumes*. Where *@Consumes* checks what the client sent via the *Content-Type* header, *@Produces* checks what the client is asking to receive back via the *Accept* header. A mismatch on that side produces 406 Not Acceptable instead of 415. In both cases Jersey rejects the request before the method body runs.

Without *@Consumes* and a proper exception mapper, a wrong *Content-Type* would reach the method body, fail during deserialisation, and bubble up as a raw 500. With the annotation in place, Jersey catches the mismatch at routing time, and the client receives a 415 with a structured JSON body rather than an unhandled server error.

Question 6

Q6. You implemented this filtering using *@QueryParam*. Contrast this with an alternative design where the type is part of the URL path (e.g., */api/v1/sensors/type/CO2*). Why is the query parameter approach generally considered superior for filtering and searching collections?

REST draws a clear line between identifying a resource and filtering one. The path identifies the resource. The query string refines what the server returns from it. */api/v1/sensors* is the sensors collection. */api/v1/sensors?type=CO2* is a filtered view of that same collection. The base resource is the same in both cases, only the subset returned changes.

Putting the filter in the path, as in */api/v1/sensors/type/CO2*, creates a problem. It implies that *type* is its own resource that contains *CO2* as a child, which is not how the domain works. Once that pattern is accepted, every other filter needs its own path too: */sensors/status/ACTIVE*, */sensors/room/LIB-301*, and so on. Combining filters then produces paths like */sensors/type/CO2/status/ACTIVE/room/LIB-301*, each of which needs its own route, its own test, and its own entry in the discovery document.

Query parameters avoid this entirely. All filters are handled by the same method and combining them is just a matter of reading more parameters. The handler on *resource/SensorResource.java* is shown below. Adding a status filter would be one extra *@QueryParam* argument, not a new endpoint:

```java
@GET
@Produces(MediaType.APPLICATION_JSON)
public List<Sensor> getSensors(@QueryParam("type") String type) {
    if (type == null || type.trim().isEmpty()) {
        return sensorService.listAll();
    }
    return sensorService.listByType(type);
}
```

Figure 8

Query parameters also handle the unfiltered case without special treatment. Omitting the *type* parameter returns the entire collection. A path segment approach has no equivalent default: returning all sensors would require a dedicated path such as */sensors/type/all*, which introduces an artificial segment that has no meaning in the domain.

The same rule applies more broadly. Path segments identify resources with genuine identity, such as a particular sensor referenced by id. Query parameters narrow, sort, or paginate a collection. Mixing the two makes the URI schema harder to reason about without any expressive gain.

Question 7

Q7. Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., `sensors/{id}/readings/{rid}`) in one massive controller class?

A sub resource locator is a method annotated with the Path annotation but without an HTTP method annotation. It returns an object whose own Path annotated methods handle the remainder of the URI. The locator on the sensor resource is shown below.

```
@Path("/{sensorId}/readings")
public SensorReadingResource readings(@PathParam("sensorId") String sensorId) {
    return new SensorReadingResource(sensorId);
}
```

Figure 9

JAX-RS dispatches to this method when a request arrives for */sensors/{sensorId}/readings*, and the returned sub resource instance takes over dispatch for the remaining path segments.

The first benefit is separation of concerns. The sensor resource is responsible for sensors, and the readings resource is responsible for the readings that belong to one sensor. A mega controller that collected both sets of methods quickly becomes the kind of 800-line file that coding style rules explicitly discourage.

The second benefit is scoped parent context. The locator passes only the sensor id string to the sub resource's constructor. Each sub resource method resolves the parent through the reading service now it needs it, and a missing parent becomes an inline 404 response. Holding the identifier rather than a captured sensor object keeps the sub resource immune to stale snapshots if another thread mutates the parent between the locator firing and the method running.

The third benefit is that the pattern scales. A reading could acquire its own annotations sub resource in a larger system simply by adding another locator, without the top-level class growing. The pattern is also friendlier to testing: the sub resource is a plain Java class with a straightforward constructor, so it can be exercised without bootstrapping the JAX-RS runtime.

Question 8

Q8. Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

The three candidate statuses make very different claims about the failure. HTTP 404 Not Found signals that the URI in the request line does not identify a resource. HTTP 400 Bad Request signals that the server cannot parse or syntactically validate the request, meaning malformed JSON, missing required fields, or a wrong type in a field. HTTP 422 Unprocessable Entity signals that the request is syntactically well formed but semantically invalid: the server understands every byte of it but cannot act on it because a domain level constraint fails.

Consider the payload below, sent as the body of a *POST /api/v1/sensors*:

```
{
  "id": "HEAT-999",
  "roomId": "NOT-A-ROOM",
  "type": "Temperature",
  "status": "ACTIVE",
  "currentValue": 19.0
}
```

Figure 10

This request fits into the third category. The target URI */sensors* exist, so 404 would misrepresent the problem: the sensors collection is present and willing to accept registrations. The payload is valid JSON with every required field of the correct type, so 400 also misrepresents it. What fails is referential integrity between the payload and existing server state: the room that the client points at was never created.

The current implementation expresses this by throwing *LinkedResourceNotFoundException* from *service/SensorService.create* and letting *LinkedResourceNotFoundExceptionMapper* render the failure as 422 with a JSON *ErrorMessage* body containing the offending *roomId*. That response communicates three facts at once: the request was understood, re sending the same bytes will not change the outcome, and the fix is either to create the referenced room first or to correct the *roomId*. A 404 response would ambiguously suggest that the sensors endpoint itself had disappeared, and a 400 response would suggest a typo or missing field that the client cannot find. The 422 removes that confusion and leaves one clear signal.

Question 9

Q9. From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

An unfiltered stack trace returned in an HTTP error response exposes information a legitimate consumer does not need. An attacker can use that information to plan further probing. The risks break down into four categories.

First, internal paths and package structure. A single frame such as *com.w2120198.csa.cw.service.RoomService.delete* reveals the student identifier used as the root package, the module code embedded in it, the three tier layer split between resource, service, and DAO, and the exact line where the failure happened.

Second, library and framework names. Frames frequently reference third party classes such as *org.glassfish.jersey* or *com.fasterxml.jackson*. Those names reveal which frameworks the application depends on, giving an attacker a starting point for targeted probing.

Third, hints at internal logic flaws. The exception class together with the line number indicates where the failure happened and often suggests why. A null pointer at a particular line narrows the guess about which variable, and crafted payloads can reach or exhaust that branch.

Fourth, database schema details. Exception messages and frame context can expose table names, column references, or query fragments that reveal how the data layer is structured.

The remedy is the global exception mapper, which declares an *ExceptionMapper<Throwable>* as the final safety net for anything that no more specific mapper has claimed. Unexpected errors are logged server side through *java.util.logging* and returned to the client as the generic body below.

```
{
  "errorMessage": "The server encountered an unexpected error.",
  "errorCode": 500
}
```

Figure 11

The response contains nothing about the internal path, the library, the line, or the JVM.